

简单总结你实现的功能

在 `os/proc.h` 头文件中的 `struct proc` 里添加了开始时间 `start_time` 和 `syscalls` 次数 `syscall_times` 两个成员变量，用于记录进程的开始时间和系统调用次数。

在 `os/syscall.c` 中实现了 `sys_task_info` 系统调用，用于获取当前系统中所有进程的信息。在 `syscall` 中添加了 `SYS_task_info` 系统调用号并统计 `syscall` 的调用次数。

在 `os/proc.c` 中的 `proc_init` 函数中初始化了 `start_time` 和 `syscall_times` 两个成员变量。在 `scheduler` 函数里进程被调度时检查是否为初次调度，如果是则记录开始时间。

问答题

- 1. 正确进入 U 态后，程序的特征还应有：使用 S 态特权指令，访问 S 态寄存器后会报错。请同学们可以自行测试这些内容（参考 [前三个测试](#)，描述程序出错行为，同时注意注明你使用的 sbi 及其版本。
 - 题目要求：在 RISC-V 架构下的 U（用户态）和 S（特权态）模式切换的行为分析。
 - 程序正确进入 U 态：
 - 程序必须正确运行在 U 态（用户态），不能直接访问 S 态（特权态）的资源。
 - 验证特权限制：
 - 测试用户态执行 S 态特权指令 或 访问 S 态寄存器 时是否触发异常，并记录异常行为。
 - 参考前述测试用例：
 - 日志中显示的 `__ch2_bad_address`、`__ch2_bad_instruction` 和 `__ch2_bad_register` 测试用例就是验证这些特性的测试程序。
 - 描述程序出错行为：
 - 需要分析这些测试用例触发的异常，以及错误日志中提供的信息。
 - 注明使用的 SBI 及版本：
 - 日志中显示使用的 `rustsbi`，版本为 `0.3.0-alpha.2`。
 - 执行与分析
 - `make test CHAPTER=2_bad` 执行后的日志

```

_____
|  _  \  | | | | /  |  | /  ||  _  \
|  |
|  |_) | | | | | (----`---| |----`| (----`| |_)
|  |
|  /  | | | | \  \  |  | \  \  |  _ <
|  |
|  \  \----.| `--' |.----) |  | |.----) |  | |_)
|  |
|  | `_.____| \_____/ |_____/ |  | |_____/ |_____/
|  |
[rustsbi] Implementation      : RustSBI-QEMU Version 0.2.0-alpha.2
[rustsbi] Platform Name       : riscv-virtio,qemu
[rustsbi] Platform SMP       : 1
[rustsbi] Platform Memory     : 0x80000000..0x88000000
```

```

[rustsbi] Boot HART : 0
[rustsbi] Device Tree Region : 0x87000000..0x87000f02
[rustsbi] Firmware Address : 0x80000000
[rustsbi] Supervisor Address : 0x80200000
[rustsbi] pmp01: 0x00000000..0x80000000 (-wr)
[rustsbi] pmp02: 0x80000000..0x80200000 (---)
[rustsbi] pmp03: 0x80200000..0x88000000 (xwr)
[rustsbi] pmp04: 0x88000000..0x00000000 (-wr)
hello wrold!
__ch2_bad_address
[ERROR 0]unknown trap: 0x0000000000000007, stval =
0x0000000000000000 sepc = 0x0000000080400012
__ch2_bad_instruction
[ERROR 0]IllegalInstruction in application, epc =
0x0000000080400012, core dumped.
__ch2_bad_register
[ERROR 0]IllegalInstruction in application, epc =
0x0000000080400012, core dumped.
ALL DONE

```

在正确进入 U 态后，我们测试了用户态对特权资源的访问限制。

- 测试用例：__ch2_bad_address
访问非法地址，触发 Store/AMO Address Misaligned 异常。
代码：

```

#include <stdio.h>
#include <unistd.h>

int main()
{
    int *p = (int *)0;
    *p = 0;
    return 0;
}

```

日志：

```

[ERROR 0]unknown trap: 0x0000000000000007, stval =
0x0000000000000000 sepc = 0x0000000080400012

```

- 异常说明：
 - trap type: 0x7 表示 Store/AMO Address Misaligned（存储/原子操作地址未对齐）异常。
 - stval: 0x0，异常发生时，stval 存储了触发异常的地址。
 - 这里 stval=0x0，说明程序试图访问了非法或未对齐的地址。
 - sepc: 0x80400012，这是导致异常的程序计数器地址，指向异常指令的位置。
- 分析：
 - 该测试用例验证用户态访问非法地址的行为。
 - 程序尝试访问 S 态或机器态专属的地址空间（或未映射的地址空间）。
 - 结果触发地址未对齐异常，这是符合预期的用户态行为。

- 测试用例: `__ch2_bad_instruction`
执行特权指令, 触发 `IllegalInstruction` 异常。

代码:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    asm volatile("sret");
    return 0;
}
```

日志:

```
[ERROR 0]IllegalInstruction in application, epc =
0x0000000080400012, core dumped.
```

- 异常说明:

- 异常类型: `IllegalInstruction` 表示执行了非法指令。
- epc: `0x80400012`, 这是导致异常的指令地址。

- 分析:

- 该测试用例验证用户态执行特权指令 (如 `sret`, `sfence.vma`) 的行为。
- 在 RISC-V 架构中, 这些特权指令只能在 S 态或更高权限下执行。
- 用户态执行这些指令会触发 `IllegalInstruction` 异常, 这是符合预期的行为。

- `sret` Supervisor Return

`sret` 是什么?

- `sret` 是 RISC-V 指令集架构中的一条特权指令, 全称为 **Supervisor Return**, 意为 **从 S 态返回到 U 态** 或其他低权限态。

`sret` 的核心功能是: 它主要用于从 **S (Supervisor) 特权模式** 返回到 **U (User) 用户模式**, 以继续运行用户态程序。

- 恢复处理器的 **程序计数器 (PC)** 和 **权限模式**, 以返回到更低权限的状态 (通常是 U 态)。
- 它通过 `sstatus` 寄存器 中的特定字段 (`SPP` 和 `SPIE`) 来决定返回地址和中断状态。

- 测试用例: `__ch2_bad_register`
访问特权寄存器, 触发 `IllegalInstruction` 异常。

代码:

```
#include <stdio.h>
#include <unistd.h>

int main()
{
    uint64 x;
    asm volatile("csrr %0, sstatus" : "=r"(x));
    return 0;
}
```

日志:

```
[ERROR 0]IllegalInstruction in application, epc =
0x0000000080400012, core dumped.
```

■ 异常说明:

- 与 `__ch2_bad_instruction` 的异常类似, 触发了 `IllegalInstruction`。
- **epc:** `0x80400012`, 指向异常指令地址。

■ 分析:

- 该测试用例验证用户态访问特权寄存器的行为。
- 用户态不允许访问特权寄存器 (如 `sstatus`, `satp` 等)。
- 程序访问这些寄存器会触发 `IllegalInstruction` 异常, 这是符合预期的行为。

- `asm volatile("csrr %0, sstatus" : "=r"(x));`

用来在 C 代码中执行特定的 RISC-V 汇编指令。它的作用是读取 RISC-V 的 `sstatus` 寄存器的值, 并将其存储到变量 `x` 中。

- 2. 请结合用例理解 [trampoline.S](#) 中两个函数 `userret` 和 `uservec` 的作用, 并回答如下几个问题:

○ `uservec`

```
.globl userret
userret:
    # userret(TRAPFRAME, pagetable)
    # switch from kernel to user.
    # usertrapret() calls here.
    # a0: TRAPFRAME, in user page table.
    # a1: user page table, for satp.

    # switch to the user page table.
    # csrw satp, a1
    # sfence.vma zero, zero

    # put the saved user a0 in sscratch, so we
    # can swap it with our a0 (TRAPFRAME) in the last step.
    ld t0, 112(a0)
    csrw sscratch, t0

    # restore all but a0 from TRAPFRAME
    ld ra, 40(a0)
```

```

ld sp, 48(a0)
ld gp, 56(a0)
ld tp, 64(a0)
ld t0, 72(a0)
ld t1, 80(a0)
ld t2, 88(a0)
ld s0, 96(a0)
ld s1, 104(a0)
ld a1, 120(a0)
ld a2, 128(a0)
ld a3, 136(a0)
ld a4, 144(a0)
ld a5, 152(a0)
ld a6, 160(a0)
ld a7, 168(a0)
ld s2, 176(a0)
ld s3, 184(a0)
ld s4, 192(a0)
ld s5, 200(a0)
ld s6, 208(a0)
ld s7, 216(a0)
ld s8, 224(a0)
ld s9, 232(a0)
ld s10, 240(a0)
ld s11, 248(a0)
ld t3, 256(a0)
ld t4, 264(a0)
ld t5, 272(a0)
ld t6, 280(a0)

# restore user a0, and save TRAPFRAME in sscratch
csrrw a0, sscratch, a0

# return to user mode and user pc.
# usertrapret() set up sstatus and sepc.
sret

```

- 作用：
 - 是从用户态（U 模式）触发异常或中断时的第一入口。
 - 用于处理用户态的陷入（trap）情况，例如系统调用（syscall）、非法指令、页错误等。
- 功能：
 1. **保存用户态上下文：**
 - 将触发 trap 时的用户寄存器状态保存到 `trapframe`（存储在 `a0` 中，指向进程的 `trapframe` 结构）。
 2. **调用内核 trap 处理函数：**
 - 将控制权转交给内核，用于进一步处理异常或中断。
 3. **在必要时切换页表：**
 - 内核可能会切换到自己的内核页表来处理 trap 情况。

○ `userret`

- 作用：
 - 是从内核态（S 模式）返回到用户态（U 模式）时的最后入口。

- 用于将执行权限从内核还给用户程序。
- 功能：
 1. 恢复用户态上下文：
 - 从 `trapframe` 中读取用户寄存器的值，并恢复。
 2. 切换到用户态页表：
 - 切换到用户程序的页表（`satp` 寄存器）。
 3. 进入用户态：
 - 使用 `sret` 指令切换到 U 模式并恢复用户态程序计数器（`sepc`）。
- L79: 刚进入 `userret` 时，`a0`、`a1` 分别代表了什么值。
 - `a0`：指向当前进程的 `trapframe`，即保存用户态上下文的结构。`trapframe` 是一个数据结构，用于保存用户态寄存器的值、程序计数器（`sepc`）、状态寄存器（`sstatus`）等内容。

在 `syscall` 中，通过

```
extern char trap_page[];

void syscall()
{
    struct trapframe *trapframe = (struct trapframe *)trap_page;
    int id = trapframe->a7, ret;
    uint64 args[6] = { trapframe->a0, trapframe->a1, trapframe->a2,
                      trapframe->a3, trapframe->a4, trapframe->a5 };
    tracef("syscall %d args = [%x, %x, %x, %x, %x, %x]", id,
          args[0],
              args[1], args[2], args[3], args[4], args[5]);
}
```

- `a1`：保存用户页表的物理地址，最终会写入 `satp` 寄存器，用于切换到用户页表。
- L87-L88: `sfence` 指令有何作用？为什么要执行该指令，当前章节中，删掉该指令会导致错误吗？

```
csrw satp, a1
sfence.vma zero, zero
```

- `sfence.vma` 指令的作用：
 - 刷新 TLB（Translation Lookaside Buffer，地址转换缓存）。
 - 保证页表切换后，旧的页表映射不会污染新的页表。
- 为什么要执行 `sfence.vma`：
 - 当切换到用户页表（`csrw satp, a1`）后，旧的虚拟地址到物理地址映射仍可能被缓存（在 TLB 中）。
 - 如果不执行 `sfence.vma` 指令，可能会导致后续使用错误的地址映射。
- 当前章节中，删除 `sfence.vma` 是否会导致错误？
 - 是的，可能导致错误：
 - 如果程序访问了旧页表映射中的地址，会导致内存错误或访问非法地址。
 - 例如，如果用户页表和内核页表的地址空间有重叠，可能出现不一致行为。
- L96-L125: 为何注释中说要除去 `a0`？哪一个地址代表 `a0`？现在 `a0` 的值存在何处？

```
# restore all but a0 from TRAPFRAME
ld ra, 40(a0)
ld sp, 48(a0)
ld t5, 272(a0)
ld t6, 280(a0)
```

- 为何要“除去 a0”？
 - 在用户态程序运行时，a0 通常存储函数的第一个参数（根据 ABI 调用约定）。
 - 但这里的 a0 已被用作 trapframe 的地址，不能直接恢复为用户态程序的值。
 - 因此，a0 需要在最后单独处理。
- 哪一个地址代表 a0？
 - trapframe 中保存了用户态的 a0 值，存储在 trapframe 的偏移量为 112 的位置。
- 现在 a0 的值存在何处？
 - 当前 a0 存储的是 trapframe 的地址。
 - 用户态的原始 a0 值存储在 trapframe 的 112(a0) 位置。
- userret：中发生状态切换在哪一条指令？为何执行之后会进入用户态？
 - 状态切换的指令：sret。
 - 为何进入用户态：
 - sret 会从 sstatus.SPP 恢复权限模式（SPP=0 表示返回到用户态）。
 - 同时，从 sepc 恢复程序计数器，从而继续执行用户态程序。
- L29：执行之后，a0 和 sscratch 中各是什么值，为什么？


```
csrrw a0, sscratch, a0
```

 - 执行之后：
 - a0：保存 sscratch 中的原始值（用户态的 a0）。
 - sscratch：保存原始的 a0 值（trapframe 的地址）。
 - 原因：
 - 交换了 a0 和 sscratch，方便后续操作中访问 trapframe。
- L32-L61：从 trapframe 第几项开始保存？为什么？是否从该项开始保存了所有的值，如果不是，为什么？

```
sd ra, 40(a0)
sd sp, 48(a0)
...
sd t5, 272(a0)
sd t6, 280(a0)
```

- 从第几项开始保存：
 - 从 trapframe 的偏移量 40 开始保存（ra 寄存器）。
 - 偏移量 40 之前保存的是异常处理需要的基本信息，如：
 - 0：内核页表地址（satp）。
 - 16：用户程序入口地址（sepc）。
- 是否保存了所有的值：
 - 保存了用户态程序的绝大多数寄存器，但没有保存 a0（单独处理）。
 - 这是因为 a0 在 trapframe 的偏移量 112 中被单独保存。
- 进入 S 态是哪一条指令发生的？

- **指令：** `csrrw a0, sscratch, a0`。

Control and Status Register Read and Write

- **原因：**

- 在用户态触发异常时，硬件会将陷入模式切换到 S 态，并将 `sscratch` 设置为 `trapframe` 的地址。
- 这条指令的目的是交换 `a0` 和 `sscratch` 的值，便于后续操作。

- L75-L76: `ld t0, 16(a0)` 执行之后，`t0` 中的值是什么，解释该值的由来？

```
ld t0, 16(a0)
jr t0
```

- **t0 的值：** 用户程序的入口地址（程序计数器 `sepc` 的值）。

- **值的由来：**

- 当用户程序发生异常时，硬件会将异常时的程序计数器（PC）保存到 `sepc`。
- `trapframe` 的偏移量 `16` 对应 `sepc` 的值。
- 这条指令将用户程序的入口地址加载到 `t0`，便于后续返回用户态继续执行。